

Дисциплина «Базы данных»

Самостоятельная работа № 1

ФИО студента	Егорушкин Иван Сергеевич
Группа	ДЭ 16-25
Дата выполнения	

1) SQL код

```
CREATE TABLE registration (  
    id_user SERIAL PRIMARY KEY,  
    name_user VARCHAR(255) NOT NULL,  
    role VARCHAR(20) NOT NULL CHECK (role IN ('student','teacher')),  
    date_reg DATE NOT NULL DEFAULT CURRENT_DATE,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE student (  
    id_student SERIAL PRIMARY KEY,  
    reg_inf BIGINT NOT NULL UNIQUE,  
    groupe VARCHAR(50),  
    FOREIGN KEY (reg_inf) REFERENCES registration(id_user) ON DELETE CASCADE  
);  
  
CREATE TABLE teacher (  
    id_teacher SERIAL PRIMARY KEY,  
    teacher_inf BIGINT NOT NULL UNIQUE,  
    requisites VARCHAR(255) NOT NULL,  
    FOREIGN KEY (teacher_inf) REFERENCES registration(id_user) ON DELETE CASCADE  
);  
  
CREATE TABLE course (  
    id_course SERIAL PRIMARY KEY,  
    name_course VARCHAR(255) NOT NULL,  
    teacher BIGINT NOT NULL,  
    price BIGINT NOT NULL CHECK (price >= 0),  
    FOREIGN KEY (teacher) REFERENCES teacher(id_teacher) ON DELETE RESTRICT  
);  
  
CREATE TABLE lesson (  
    id_lesson SERIAL PRIMARY KEY,  
    name_lesson VARCHAR(255) NOT NULL,  
    course BIGINT NOT NULL,  
    FOREIGN KEY (course) REFERENCES course(id_course) ON DELETE CASCADE  
);  
  
CREATE TABLE course_progress (  
    course BIGINT NOT NULL,
```

```

        student BIGINT NOT NULL,
        date_reg DATE NOT NULL DEFAULT CURRENT_DATE,
        course_status TEXT NOT NULL CHECK (course_status IN
('enrolled','in_progress','completed','failed')),
        PRIMARY KEY (course, student),
        FOREIGN KEY (course) REFERENCES course(id_course) ON DELETE CASCADE,
        FOREIGN KEY (student) REFERENCES student(id_student) ON DELETE CASCADE
    );

CREATE TABLE payments (
    id_pay SERIAL PRIMARY KEY,
    course BIGINT NOT NULL,
    teacher BIGINT NOT NULL,
    dates DATE NOT NULL DEFAULT CURRENT_DATE,
    amount BIGINT NOT NULL CHECK (amount >= 0),
    FOREIGN KEY (course) REFERENCES course(id_course) ON DELETE CASCADE,
    FOREIGN KEY (teacher) REFERENCES teacher(id_teacher) ON DELETE RESTRICT
);

-- + 2 таблицы отзывы и дипломы преподавов
CREATE TABLE review (
    student BIGINT NOT NULL,
    course BIGINT NOT NULL,
    rating SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment TEXT,
    date_review DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (student, course),
    FOREIGN KEY (student) REFERENCES student(id_student) ON DELETE CASCADE,
    FOREIGN KEY (course) REFERENCES course(id_course) ON DELETE CASCADE
);

CREATE TABLE certificate (
    id_cert SERIAL PRIMARY KEY,
    student BIGINT NOT NULL,
    course BIGINT NOT NULL,
    issued_at DATE NOT NULL DEFAULT CURRENT_DATE,
    cert_number VARCHAR(50) UNIQUE NOT NULL,
    FOREIGN KEY (student) REFERENCES student(id_student) ON DELETE CASCADE,
    FOREIGN KEY (course) REFERENCES course(id_course) ON DELETE RESTRICT
);

-- Список курсов преподавателя
CREATE INDEX idx_course_teacher ON course(teacher);

-- Список уроков курса (сортировка по id)
CREATE INDEX idx_lesson_course ON lesson(course, id_lesson);

-- Статус прохождения конкретного курса
CREATE INDEX idx_cp_course_status ON course_progress(course, course_status);

-- Средний рейтинг курса

```

```

CREATE INDEX idx_review_course_rating ON review(course, rating);

-- Выплаты преподавателю за период
CREATE INDEX idx_payments_teacher_date ON payments(teacher, dates);

-- Сертификаты студента
CREATE INDEX idx_certificate_student ON certificate(student);

INSERT INTO registration (name_user, role, email, password) VALUES
('Иван Егорушкин', 'student', 'не_валите_пж@please.me', 'hash1'),
('Ариша Кучеровская', 'student', 'ее_можно@joke.xd', 'hash2'),
('Алерсандр Петраки', 'teacher', 'killer@die.tim', 'hash3');

INSERT INTO student (reg_inf, groupe) VALUES (1, 'ДЭ-16-25'), (2, 'ДЭ-16-25');
INSERT INTO teacher (teacher_inf, requisites) VALUES (3, 'С6ер 1234');

INSERT INTO course (name_course, teacher, price) VALUES ('PostgreSQL', 1, 10000);
INSERT INTO lesson (name_lesson, course) VALUES ('Intro', 1);
INSERT INTO course_progress (course, student, course_status) VALUES (1, 1,
'enrolled');
INSERT INTO review (student, course, rating) VALUES (1, 1, 5);
INSERT INTO payments (course, teacher, amount) VALUES (1, 1, 8000);
INSERT INTO certificate (student, course, cert_number) VALUES (1, 1, 'CERT-2025-
001');

-- Нарушение CHECK
DO $$
BEGIN
    INSERT INTO review (student, course, rating) VALUES (2, 1, 10);
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'Ошибка: оценка курса должна быть в диапазоне от 1 до 5. '
        'Пользователь передал некорректное значение. Текст ошибки:
%', SQLERRM;
END;
$$;

-- Нарушение FOREIGN KEY
DO $$
BEGIN
    INSERT INTO course_progress (course, student, course_status) VALUES (999, 1,
'enrolled');
EXCEPTION
    WHEN foreign_key_violation THEN
        RAISE NOTICE 'Ошибка: нельзя записать студента на несуществующий курс. '
        'Проверьте, что курс создан в системе. Текст ошибки: %',
SQLERRM;
END;
$$;

```

```

-- Нарушение UNIQUE
DO $$
BEGIN
    INSERT INTO registration (name_user, role, email, password) VALUES
('Дубликат', 'student', 'не_валите_пж@please.me', 'hashX');
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'Ошибка: пользователь с таким email уже зарегистрирован. '
        'Используйте другой email или восстановите пароль. Текст
ошибки: %', SQLERRM;
END;
$$;

-- Нарушение NOT NULL
DO $$
BEGIN
    INSERT INTO lesson (name_lesson, course) VALUES (NULL, 1);
EXCEPTION
    WHEN not_null_violation THEN
        RAISE NOTICE 'Ошибка: у урока обязательно должно быть название. '
        'Название не может быть пустым. Текст ошибки: %', SQLERRM;
END;
$$;

-- Нарушение PRIMARY KEY
DO $$
BEGIN
    INSERT INTO course_progress (course, student, course_status) VALUES (1, 1,
'completed');
EXCEPTION
    WHEN unique_violation THEN
        RAISE NOTICE 'Ошибка: студент уже записан на этот курс (нарушение PRIMARY
KEY). '
        'Нельзя создать дублирующую запись о прогрессе. Текст
ошибки: %', SQLERRM;
END;
$$;

```

2) Документация нарушений ограничений

№	Ограничени е	Выполняемый запрос (SQL)	Сообщение СУБД	Понятное сообщение для пользователя	Как исправить
1	CHECK	INSERT INTO review (student, course, rating) VALUES (2, 1, 10);	new row for relation "review" violates check constraint "review_rating_check"	«Оценка курса должна быть целым числом от 1 до 5»	Указать корректное значение: rating BETWEEN 1 AND 5

2	FOREIGN KEY	INSERT INTO course_progress (course, student, course_status) VALUES (999, 1, 'enrolled');	insert or update on table "course_progress" violates foreign key constraint "fk_cp_course"	«Нельзя записать студента на несуществующий курс. Сначала создайте курс»	Указать существующий course.id_course
3	UNIQUE	INSERT INTO registration (name_user, role, email, password) VALUES ('Дубликат', 'student', 'ivan@edu.ru', 'hashX');	duplicate key value violates unique constraint "registration_email_key"	«Пользователь с таким email уже зарегистрирован»	Использовать другой email или выполнить восстановление пароля
4	NOT NULL	INSERT INTO lesson (name_lesson, course) VALUES (NULL, 1);	null value in column "name_lesson" of relation "lesson" violates not-null constraint	«У урока обязательно должно быть название»	Передать непустое значение name_lesson
5	PRIMARY KEY	INSERT INTO course_progress (course, student, course_status) VALUES (1, 1, 'completed');	duplicate key value violates unique constraint "course_progress_pkey"	«Студент уже записан на этот курс — дублирование записи о прогрессе недопустимо»	Обновить существующую запись (UPDATE) вместо INSERT